Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VI

```
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : [],
    'D' : [],
    'E' : ['F'],
    'F' : []
}
# Menambahkan kode program dari materi DFS Flow
graph_val = {"A": 10, 'B': 12, 'C': 3, 'D': 4, 'E': -6, 'F': -5}
```

```
visited = set() # Set to keep track of visited nodes.
```

```
leaf = set() # Menambahkan kode program dari materi DFS Flow
```

```
def dfs(visited, graph, node):
    if node not in visited:
        visited.add(node)
        # Menambahkan kode program dari materi DFS Flow
        if len(graph[node]) == 0:
            leaf.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

```
def leaf_sum(leaf):
    jmlh = 0
    for leaf_node in leaf:
        leaf_val = graph_val[leaf_node]
        jmlh = jmlh + leaf_val

    return jmlh
```

```
dfs(visited, graph, 'A')
```

```
leaf_sum(leaf)
```

```
2
```

Logika Program DFS Leaf Sum

1. Struktur Data

Program mendefinisikan dua hal:

- `graph` — representasi graf berarah (adjacency list), menunjukkan siapa tetangga tiap node
- `graph_val` — nilai/bobot setiap node

2. Fungsi `dfs(visited, graph, node)`

Ini adalah inti program dengan logika rekursif:

1. **Cek apakah node sudah dikunjungi** — jika sudah, fungsi langsung berhenti (mencegah loop tak terbatas)
2. **Tandai node sebagai visited** dengan `visited.add(node)`
3. **Cek apakah node adalah leaf** — jika `graph[node]` kosong (tidak punya tetangga), masukkan ke set `leaf`
4. **Rekursi ke semua tetangga** — panggil `dfs()` untuk setiap node yang terhubung

3. Fungsi `leaf_sum(leaf)`

Setelah DFS selesai, fungsi ini menjumlahkan nilai semua leaf node yang ditemukan:

- Iterasi semua node dalam set `leaf`
- Ambil nilai dari `graph_val[leaf_node]`

- Akumulasi ke variabel `jmlh`

4. Hasil Eksekusi

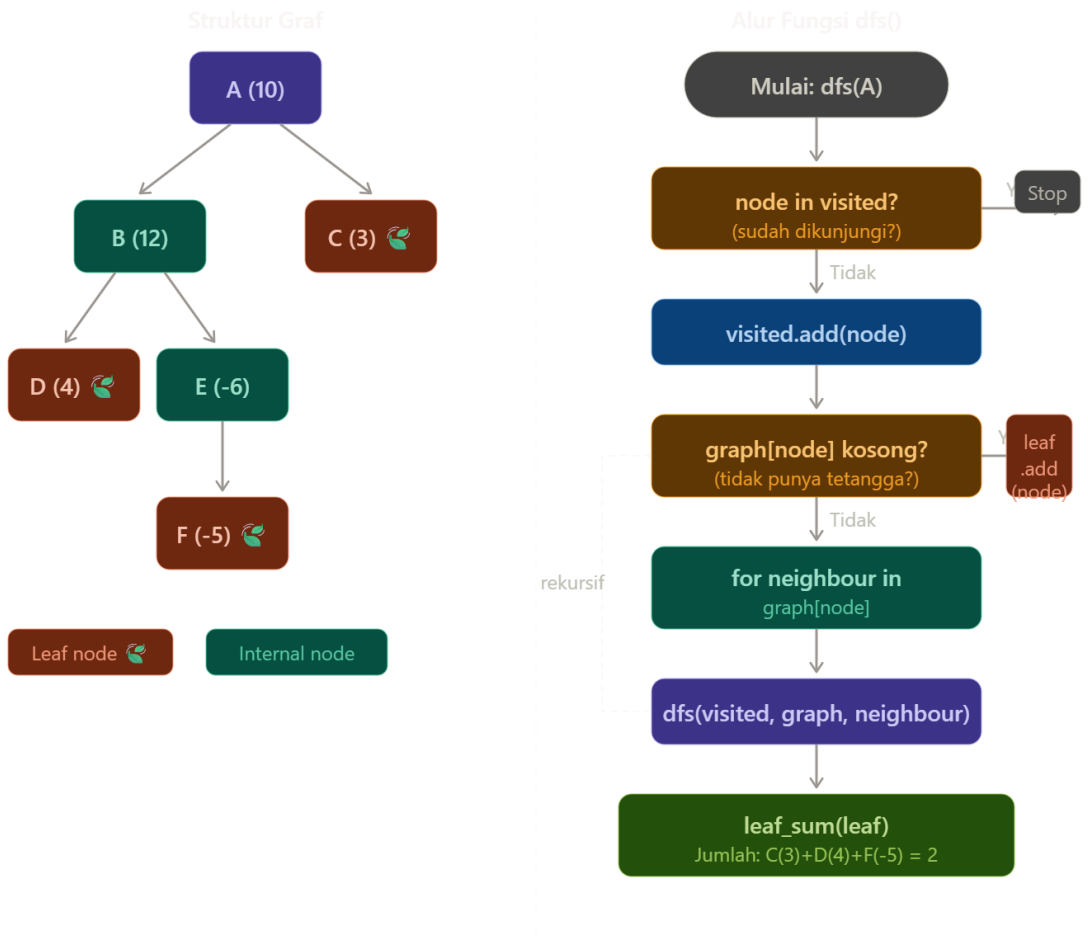
Leaf node yang ditemukan adalah **C, D, F** (tidak memiliki tetangga). Hasilnya:

```
C(3) + D(4) + F(-5) = 2
```

5. Urutan Pemanggilan DFS

```
dfs(A) → dfs(B) → dfs(D) [leaf!]
                → dfs(E) → dfs(F) [leaf!]
       → dfs(C) [leaf!]
```

> **Catatan penting:** Karena `visited` dan `leaf` adalah variabel global (`set()`), pastikan di-reset ke `set()` baru jika ingin menjalankan DFS ulang dari awal pada graf yang berbeda.



Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/

Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VI

```
# A binary tree node
class Node:
    # Constructor to create a new node
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
# Compute the "maxDepth" of a tree -- the number of nodes
# along the longest path from the root node down to the farthest leaf node
def maxDepth(node):
    if node is None:
        return -1 ;

    else :

        # Compute the depth of each subtree
        lDepth = maxDepth(node.left)
        rDepth = maxDepth(node.right)

        # Use the larger one
        if (lDepth > rDepth):
            return lDepth+1
        else:
            return rDepth+1
```

```
# Driver program to test above function
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
```

```
print ("Height of tree is %d" %(maxDepth(root)))

# This code is contributed by Amit Srivastav
```

Logika Program Graph Height (maxDepth)

1. Class `Node`

Merepresentasikan satu simpul pada binary tree. Setiap node menyimpan tiga hal: `data` (nilai), `left` (anak kiri), dan `right` (anak kanan). Nilai awal `left` dan `right` adalah `None` (belum punya anak).

2. Fungsi `maxDepth(node)` — Rekursif

Fungsi ini menghitung **kedalaman maksimum** pohon, yaitu panjang jalur terpanjang dari root ke leaf.

Base case — jika `node is None`, kembalikan `-1`. Ini terjadi saat kita melewati leaf node (anak dari leaf adalah `None`).

Rekursif case:

1. Hitung kedalaman subtree kiri → `lDepth = maxDepth(node.left)`
2. Hitung kedalaman subtree kanan → `rDepth = maxDepth(node.right)`
3. Ambil yang lebih besar, tambah 1 → `max(lDepth, rDepth) + 1`

3. Struktur Pohon yang Diuji

```
     1          ← depth 0
    / \
   2   3        ← depth 1
  / \
 4   5          ← depth 2
```

4. Jejak Rekursi

Panggilan	lDepth	rDepth	Return
`maxDepth(4)`	-1	-1	0
`maxDepth(5)`	-1	-1	0
`maxDepth(3)`	-1	-1	0
`maxDepth(2)`	0	0	1
`maxDepth(1)`	1	0	**2**

5. Hasil

```
Height of tree is 2
```

> **Mengapa return `-1` bukan `0`?** Karena kita menghitung **edge** (jumlah tingkat), bukan node. Leaf node itu sendiri bernilai 0, dan `None` (di bawah leaf) bernilai -1, sehingga saat ditambah 1 hasilnya tepat 0 untuk leaf.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/